

Does the Source of Carrier Image Affect Steganographic Detectability?

Midway Project Proposal

Project 2.2, Department of Advanced Computing Sciences

Abdul Moiz Akbar Daria Gjonbalaj Nico Müller-Späth
Jimena Narvaez del Cid David Wicker Nikolas Zouros

Maastricht University · February 2026

1. Motivation and Problem Statement

Steganography is the practice of hiding information inside ordinary media so that the existence of the message is concealed, not just its contents [1, 2]. That makes it relevant in both legitimate and adversarial settings. In either case, however, embedding a message is only useful if the modified file does not become easy to detect. For that reason, steganalysis is not separate from the problem, but part of it.

That detectability depends on more than the embedding method alone. The carrier image also matters [1, 2]. Steganalysis detectors look for statistical irregularities introduced into pixel values or frequency coefficients, and how visible those irregularities are depends on the carrier’s original statistics [9, 21]. The same payload and the same embedding method can therefore be easier to detect in one image than in another.

This becomes especially relevant once we move beyond ordinary photographs. Most steganalysis work assumes a camera image as the carrier, which is reasonable because camera photographs have well-studied noise patterns and JPEG traces. Photorealistic diffusion models such as Stable Diffusion [5], however, are now widely available and generate images through a process that does not follow a camera pipeline. Corvi et al. [15] showed that diffusion outputs have their own statistical fingerprints, which raises a natural question: do detectors that work on photographs behave the same way on synthetic carriers?

Our central question is therefore whether steganalysis detectors validated on photographs remain equally effective when the carrier is ML-generated. If synthetic carriers make detection harder or easier, then carrier choice alone could change the risk of detection. We found little prior work that tests classical steganalysis across carrier types in a controlled setting. Gao et al. [8] study coverless generative steganography, where the generated image *is* the secret rather than a carrier with an embedded payload. Méreur et al. [25] come closer to our question, but they use adaptive embedding (HILL, UNIWARD, MiPOD) and report P_E rather than AUC for classical LSB-style pipelines (see Section 6).

We address this gap by comparing detectability across three carrier sources: real photographs, SDXL 1.0 outputs, and PixArt- α outputs. All three sources go through the same embedding, payload, and detector settings. The design covers 2 embedding methods $\times$ 3 payload levels $\times$ 2 encryption conditions over 1,500 images (500 per

source). We use training-free statistical detectors matched to each embedding branch. Since these detectors have no learned parameters, AUC differences between carrier groups should reflect carrier statistics rather than training mismatch. The synthetic images use the same captions as the real photographs, but they are not pixel-level matches, so the comparison is best understood as a domain-shift study rather than a tightly controlled source-only test.

Contributions. We build a caption-matched carrier dataset across three sources (real, SDXL, PixArt- α) and evaluate training-free steganalysis detectors on all of them under the same embedding, payload, and encryption settings. The main result is an AUC-based comparison using DeLong tests with Bonferroni–Holm correction to test whether carrier source changes detectability in both the spatial and frequency branches.

2. Research Questions

RQ1: Primary (Carrier Source, Real vs. ML)

Does carrier source (real photographs vs. ML-generated images) affect steganographic detectability under matched embedding settings? *We pool ML-A and ML-B into one ML group for this comparison, but we also report per-generator breakdowns in case the two generators move in opposite directions.*

RQ2: Primary (Generator Effect Within ML)

Within ML-generated carriers, does the choice of generator (ML-A vs. ML-B) affect detectability?

RQ3: Exploratory (Payload Interaction)

Does payload size change the detectability gap between carrier sources?

RQ4: Exploratory (Embedding Branch)

Do the spatial branch (LSB+PNG) and frequency branch (DCT-LSB+JPEG) show different carrier-source effects? *Each branch bundles an embedding algorithm with its file format, so any cross-branch difference reflects the full pipeline, not just the embedding rule alone.*

RQ5: Verification (Encryption Invariance)

Does AES-256-CBC encryption of the payload affect detectability? *Encrypted payloads should look like random bitstreams, so we expect no difference. A positive result here would suggest the detector is reacting to payload structure rather than to embedding distortion itself.*

Verification details for each RQ are specified in Section 4.

3. Chosen Approaches

3.1 Datasets

Real images (500 total). We draw from **COCO** [16] (300 images) and **Flickr30k** [17] (200 images) for their detailed captions. We exclude captions that are too vague to describe a scene. All images are converted to grayscale ($\text{luma } 0.299R + 0.587G + 0.114B$), resized to 512×512 , and stored as 8-bit PNG for the LSB branch and JPEG Q=95 for the DCT branch. Grayscale keeps the pipeline single-channel and closer to BOSSBase [19].

ML-generated images (1,000 total). We reuse the same captions with two generators via **diffusers** [24]: **ML-A** (500 images) with Stable Diffusion XL 1.0 [6] and **ML-B** (500 images) with PixArt- α [7]. Each caption is used once per model, so the synthetic images share subject matter with the originals but not pixel-level structure.

Quality control. We screen generated images for two failure modes: **(1)** saturation, where the grayscale mean is above 245 or below 10, and **(2)** semantic failure, where the image is near-uniform or the main subject is missing. Semantic failures are checked by manual inspection on a 10% holdout per generator. We considered BRISQUE [22], but it was designed for camera images and is not reliable for AI-generated content [23]. Failed images are regenerated with a new seed; if that fails too, we replace the caption with the next unused one to keep $N=500$ per source group.

3.2 Embedding Methods

LSB substitution (spatial domain) replaces the k least significant bits of each pixel with message bits in row-major order. We define three payload levels by the fraction of available positions used:

Table 1: Payload levels matched by fill rate. Both branches use $k=1$ LSB replacement.

Level	Fill	Spatial bpp	DCT fill
Low	25%	0.25	25% of non-zero ACs
Medium	50%	0.50	50% of non-zero ACs
High	75%	0.75	75% of non-zero ACs
<i>Bit-depth sensitivity (reported separately, not in RQ3 analysis):</i>			
BD-Sens	75% ($k=2$)	1.50	n/a

The three main payload levels all use $k=1$, so RQ3 isolates fill-rate effects. The BD-Sens condition ($k=2$, 1.50 bpp) is a spatial-only auxiliary check and is excluded from RQ3 because it changes bit-depth and fill rate together.

DCT-LSB embedding (frequency domain) operates on JPEG carriers (Q=95). Following JSteg [10, 13], message bits replace the least significant bit of non-zero quantized AC DCT coefficients in row-major block order. We skip zero-valued and DC coefficients to reduce distortion and obvious histogram artifacts.

Implementation pipeline. DCT-LSB operates directly on quantized integer DCT coefficients via `jpegio`. Modified coefficients are re-entropy-coded with the same Q=95 table and without re-quantization to avoid double-quantization distortion.

3.3 Encryption

We test two payload conditions, plain and AES-256-CBC, in both branches.

3.4 Image-Quality Tests (Quality Control)

For each stego/cover pair we compute PSNR, SSIM, and FSIM [20] to rule out trivial quality loss.

3.5 Steganalysis Detectors

All primary detectors are training-free statistical tests, reimplemented in Python from the original publications. They add no learned-parameter bias, though AUC can still vary with carrier properties such as histogram shape and noise level. Because both branches use LSB replacement, the χ^2 attack applies to both, so each branch has at least two independent detectors.

Table 2: Steganalysis detector overview. All primary detectors are training-free.

Detector	Principle	Domain
RS Analysis [9]	Regular/Singular groups	Spatial
χ^2 attack [10]	Pairs-of-values frequency	Spatial
Sample Pairs [11]	Trace multisets	Spatial
χ^2 attack (DCT) [10]	Pairs-of-values on DCT coeff.	DCT
Calibration χ^2 [13]	Crop-recompress reference	DCT

Spatial-branch detectors. **RS analysis** [9] tracks changes in Regular and Singular pixel groups under flipping masks. **The χ^2 attack** [10] tests whether adjacent value pairs ($2k, 2k+1$) move toward equal frequency, a signature of LSB replacement. **Sample Pairs (SP) analysis** [11] uses trace multisets from pixel pairs as a third independent test. All three assume row-major embedding.

DCT-branch detectors. The **χ^2 attack (DCT)** [10] applies the same pairs-of-values idea to quantized DCT coefficients. **Calibration χ^2** [13] compares the stego histogram with a non-block-aligned crop recompressed at Q=95, which serves as a reference with reduced embedding artifacts.

Extension (time-permitting): SRNet [18], retrained on BOSSBase [19] images embedded with spatial LSB and DCT-LSB. This would add a trained detector to both branches and be reported separately.

3.6 Limitations

500 images per source group should give enough power for medium-to-large effects (AUC differences ≥ 0.05), but smaller effects may still go unnoticed. We test only two generator families, and ML-A and ML-B differ in both training data and architecture, so RQ2 cannot isolate the cause of any difference. The spatial branch has three detectors while the DCT branch has two, because fewer training-free methods exist for the DCT domain. Grayscale conversion also removes colour information that could help separate carrier sources.

4. Experiments

Statistical protocol: Exp. 1–2 are **confirmatory** (Bonferroni–Holm, $\alpha = 0.05$). Exp. 3–4 are **exploratory** and reported with AUC differences and 95% confidence intervals. Exp. 5 is a **verification** check where we expect no effect. A *stratum* is a detector–embedding–payload combination. We use the DeLong test [14] because the same images pass through all detectors, so AUC estimates within a stratum are correlated.

Exp. 1 (RQ1): Carrier Source, real vs. pooled ML. Apply all detectors per branch (spatial: RS, χ^2 , SP; DCT: χ^2 , calibration χ^2). For each stratum (detector, method, payload), compare real vs. pooled ML AUC via DeLong test and report differences with 95% CIs.

Exp. 2 (RQ2): Generation-model effect within ML. For each stratum (detector, method, payload), compare AUC between caption-matched ML-A and ML-B with the DeLong test [14]. Report detector-specific AUC differences with 95% confidence intervals.

Exp. 3 (RQ3): Payload interaction. Plot AUC and source-contrast gaps across Low, Medium, and High payload levels (all $k=1$; LSB bpp: 0.25, 0.50, 0.75; DCT fill: 25%, 50%, 75% of non-zero ACs). Report source $\times$ payload interaction with effect sizes. Analyse BD-Sens ($k=2$, 1.50 bpp) separately to test whether a second bit plane amplifies carrier-source differences.

Exp. 4 (RQ4): Embedding-branch interaction. Compare carrier-source AUC gaps between the spatial branch (LSB+PNG) and the frequency branch (DCT-LSB+JPEG). Because each branch bundles embedding with its codec, cross-branch differences reflect the full pipeline rather than the embedding rule alone. Report source $\times$ branch interaction via AUC differences with 95% CIs.

Exp. 5 (RQ5): Encryption invariance check. Compare AUC between plain and AES-256-CBC conditions per source group, branch, and detector. Encrypted payloads should behave like random bits, so we expect no AUC change. This is a sanity check on the pipeline. Report the encryption main effect and the source $\times$ encryption interaction with 95% CIs.

5. Prototype

Vertical prototype: We first validate each component in isolation on a small test subset: spatial LSB with RS/ χ^2 /SP, and DCT-LSB with χ^2 /calibration χ^2 .

Horizontal prototype: We then run the full pipeline end-to-end on a 60-image subset (20 real + 20 ML-A + 20 ML-B) at medium payload to check that the interfaces and outputs work together before scaling to 1,500 images.

6. Related Work

6.1 Classical Embedding and Detection

Most steganographic research uses photographic carriers with spatial- or frequency-domain embedding [1, 3]. A standard baseline is LSB substitution: replacing the least significant bit of pixel intensities in the spatial domain or of quantized DCT coefficients in JPEG/JSteg-like settings [10, 13]. We keep LSB replacement in both domains to stay comparable with existing steganalysis work.

For detection, RS analysis [9], the χ^2 attack [10], and Sample Pairs analysis [11] give three independent training-free tests for LSB-type embedding. For JPEG, Fridrich et al. [13] introduced a calibration-based test that recompresses a non-block-aligned crop. Trained detectors such as SRM [4] and DCTR [12] often achieve higher accuracy, but they require larger feature pipelines and would introduce training-set bias into our comparison.

6.2 AI-Generated Carriers and Closest Prior Work

Gao et al. [8] use generative models to encode steganographic secrets as coverless shares, where the generated image *is* the secret. That differs from our setting, where we embed a payload into an existing carrier. Méreur et al. [25] are the closest to our setup because they also test AI-generated images as steganographic carriers and show that synthetic covers can exploit cover-source mismatch. Their setup still differs from ours because they use adaptive embedding (HILL, UNIWARD, MiPOD), report P_E rather than AUC, and frame the study as exploratory rather than hypothesis-driven. Mallet et al. [26] give a broader survey of cover-source mismatch. Our study instead uses classical LSB embedding with training-free detectors and compares AUC through DeLong tests between real photographs and caption-matched diffusion outputs.

6.3 Cross-Domain and Synthetic-Image Forensics

Most cross-domain steganalysis work has focused on camera-to-camera shifts. Giboulot et al. [21] showed that detectors trained on one camera source lose accuracy on images from a different camera. Our setting extends that idea to a larger shift: from camera photographs to latent-diffusion outputs, which do not contain the same sensor noise or JPEG history that many steganalysis methods rely on. Corvi et al. [15] showed that diffusion-generated images have distinct statistical fingerprints, giving us a concrete reason to expect different detector behaviour on synthetic carriers.

6.4 Classical vs. Deep Steganalysis

Deep-learning detectors like SRNet [18] can reach higher accuracy, but they are more expensive and harder to interpret. We use classical detectors for the primary analysis because the project focuses on steganography and cryptography rather than building deep models. We are not proposing a new embedding method or detector. Our goal is to check whether classical steganalysis still works when the carrier comes from a diffusion model.

7. Relation to Curriculum

The encryption and threat-modelling parts connect to **Computer Security**. Generating carrier images with SDXL and PixArt- α and comparing detectors across domains draws on **AI and Machine Learning**. The pipeline design uses **Software Engineering and Architectures**. DeLong tests and multiple-comparison correction come from **Statistics**.

8. Planning

Gantt charts for Phase 2 (implementation) and Phase 3 (completion/reporting) are in Appendix A.

9. Minimal Passing Requirements

The study must implement one encryption scheme, two embedding approaches (spatial and frequency domain), and matching detectors. It must also provide defensible answers to RQ1 (carrier-source effect: real vs. ML) and RQ2 (generation-model effect within ML).

References

- [1] F. A. P. Petitcolas, R. J. Anderson, and M. G. Kuhn, "Information hiding: a survey," *Proc. IEEE*, vol. 87, no. 7, pp. 1062–1078, 1999.
- [2] A. Cheddad, J. Condell, K. Curran, and P. McKeivitt, "Digital image steganography: Survey and analysis of current methods," *Signal Process.*, vol. 90, no. 3, pp. 727–752, 2010.
- [3] M. Hussain, A. W. A. Wahab, Y. I. B. Idris, A. T. S. Ho, and K. H. Jung, "Image steganography in spatial domain: A survey," *Signal Process.: Image Commun.*, vol. 65, pp. 46–66, 2018.
- [4] J. Fridrich and J. Kodovský, "Rich models for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 3, pp. 868–882, 2012.
- [5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE CVPR*, pp. 10674–10685, 2022.
- [6] D. Podell et al., "SDXL: Improving latent diffusion models for high-resolution image synthesis," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2307.01952.
- [7] J. Chen et al., "PixArt- α : Fast training of diffusion transformer for photorealistic text-to-image synthesis," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.00426.
- [8] K. Gao, C.-C. Chang, J.-H. Horng, and I. Echizen, "Steganographic secret sharing via AI-generated photorealistic images," *EURASIP J. Wireless Commun. Netw.*, vol. 2022, art. 119, 2022. DOI: 10.1186/s13638-022-02190-8.
- [9] J. Fridrich, M. Goljan, and R. Du, "Reliable detection of LSB steganography in color and grayscale images," *IEEE Multimedia*, vol. 8, no. 4, pp. 22–28, 2001.
- [10] A. Westfeld and A. Pfitzmann, "Attacks on steganographic systems," in *Proc. 3rd Int. Workshop Information Hiding*, LNCS 1768, pp. 61–76, 1999.
- [11] S. Dumitrescu, X. Wu, and Z. Wang, "Detection of LSB steganography via sample pair analysis," *IEEE Trans. Signal Process.*, vol. 51, no. 7, pp. 1995–2007, 2003.
- [12] V. Holub and J. Fridrich, "Low-complexity features for JPEG steganalysis using undecimated DCT," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 2, pp. 219–228, 2015.
- [13] J. Fridrich, M. Goljan, and D. Hoge, "New methodology for breaking steganographic techniques for JPEGs," in *Proc. SPIE Electronic Imaging*, vol. 5020, pp. 143–155, 2003.
- [14] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach," *Biometrics*, vol. 44, no. 3, pp. 837–845, 1988.
- [15] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano, and L. Verdoliva, "On the detection of synthetic images generated by diffusion models," in *Proc. IEEE ICASSP*, pp. 1–5, 2023.
- [16] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, LNCS 8693, pp. 740–755, 2014.
- [17] P. Young, A. Lai, M. Hodosh, and J. Hockenmaier, "From image descriptions to visual denotations: New similarity metrics for semantic inference over event descriptions," *Trans. Assoc. Comput. Linguist.*, vol. 2, pp. 67–78, 2014.
- [18] M. Boroumand, M. Chen, and J. Fridrich, "Deep residual network for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 5, pp. 1181–1193, 2019.
- [19] P. Bas, T. Filler, and T. Pevný, "Break our steganographic system: The ins and outs of organizing BOSS," in *Proc. 13th Int. Conf. Information Hiding*, LNCS 6958, pp. 59–70, 2011.
- [20] L. Zhang, L. Zhang, X. Mou, and D. Zhang, "FSIM: A feature similarity index for image quality assessment," *IEEE Trans. Image Process.*, vol. 20, no. 8, pp. 2378–2386, 2011.
- [21] Q. Giboulot, R. Cogranne, D. Borghys, and P. Bas, "Effects and solutions of cover-source mismatch in image steganalysis," *Signal Process.: Image Commun.*, vol. 86, art. 115888, 2020.
- [22] A. Mittal, A. K. Moorthy, and A. C. Bovik, "No-reference image quality assessment in the spatial domain," *IEEE Trans. Image Process.*, vol. 21, no. 12, pp. 4695–4708, 2012.
- [23] C. Li et al., "AIGIQA-20K: A large database for AI-generated image quality assessment," in *Proc. IEEE/CVF CVPR Workshops (NTIRE)*, pp. 6327–6336, 2024.
- [24] P. von Platen et al., "diffusers: State-of-the-art diffusion models," Hugging Face, Inc., 2022. <https://github.com/huggingface/diffusers>
- [25] A. Méreur, A. Mallet, and R. Cogranne, "Are deepfakes a game-changer in digital images steganography leveraging the cover-source mismatch?" in *Proc. 19th Int. Conf. Availability, Reliability and Security (ARES)*, 9 pages, 2024. DOI: 10.1145/3664476.3670893.
- [26] A. Mallet, M. Beneš, and R. Cogranne, "Cover-source mismatch in steganalysis: Systematic review," *EURASIP J. Inf. Security*, vol. 2024, art. 26, 2024. DOI: 10.1186/s13635-024-00171-6.

A. Project Gantt Charts

Phase 2: Implementation

Period 5: 30 March – 15 May 2026 (7 weeks)

WBS	Task	Start	End	Days	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7
1	Data Construction										
1.1	Dataset collection (COCO/Flickr30k with captions)	30.03	10.04	10							
1.2	ML image generation (SDXL + PixArt- α)	30.03	10.04	10							
1.3	Heuristic quality screening & dual-format normalisation	07.04	17.04	8							
2	Steganography Implementation										
2.1	LSB embedding pipeline (sequential, $k=1, 2$, PNG)	07.04	17.04	8							
2.2	DCT-LSB embedding pipeline (JPEG Q=95)	07.04	17.04	8							
2.3	AES-256-CBC payload encryption	14.04	17.04	4							
3	Detection & Analysis										
3.1	RS Analysis + χ^2 detector	14.04	24.04	8							
3.2	Sample Pairs detector & calibration χ^2 setup	14.04	01.05	14							
3.3	Image quality metrics (PSNR/SSIM/FSIM)	21.04	01.05	8							
3.4	Encryption-effect experiments (RQ5)	28.04	08.05	8							
3.5	Statistical analysis & effect-size reporting	28.04	08.05	8							
4	Writing & Revision										
4.1	Visualisations + report writing	04.05	15.05	8							
4.2	Buffer / final revision	11.05	15.05	5							

■ Data ■ Steganography ■ Detection ■ Evaluation ■ Analysis ■ Writing

Milestones: **M1** (end Wk 2): Dataset ready **M2** (end Wk 3): Embedding complete **M3** (end Wk 5): Detection + metrics complete **M4** (end Wk 7): Final report submitted

Phase 3: Completion

Project Period 6: 25 May – 12 June 2026 (3 weeks)

WBS	Task	Start	End	Days	Wk 1	Wk 2	Wk 3
1.1	Complete remaining implementation	25.05	05.06	10			
1.2	Verify results & rerun experiments	25.05	05.06	10			
1.3	Statistical verification & effect sizes	01.06	05.06	5			
2.1	Presentation slides	01.06	08.06	6			
2.2	Poster	05.06	10.06	4			
2.3	Final paper write-up	01.06	12.06	10			
2.4	Buffer / revision / submission	10.06	12.06	3			

■ Implementation ■ Verification ■ Analysis ■ Slides ■ Poster ■ Paper

Milestones: **M5** (end Wk 1): Implementation finalised **M6** (end Wk 2): Results verified, slides ready **M7** (end Wk 3): All deliverables submitted